

Chapter 6: Web Application Deployment and Modern Trends

By Er. Ukesh Thapa

Syllabus

6.1 Full-stack development

6.2 Testing and QA

6.3 DevOps, continuous integration (CI) and continuous delivery (CD)

6.4 Progressive web apps (PWAs), responsive design and usability

Total Marks: 7

Full Stack Development Overview

- ❖ The development of both front-end (client-side) and back-end (server-side) portions of web applications.
- ❖ They handle the complete development process, from designing user interfaces to managing databases and server-side functions, making them versatile in building end-to-end applications.

Technology Related to Full Stack Development

- ❖ Front-end Development:
 - It is the visible part of website or web application which is responsible for user experience.
 - The user directly interacts with the front end portion of the web application or website.
 - Front-end Technologies: HTML, CSS, and Javascript
 - Front End Libraries and Frameworks: Angular js, React Js, Bootstrap, SASS, etc.

Technology Related to Full Stack Development

❖ **Back-end Development:**

- It refers to server-side development focused on how a website functions.
- It manages the database through queries and APIs triggered by client-side commands.
- Languages used: PHP, C++, Java, JavaScript, Python
- Frame work: Dot Net, Laravel, Spring Boot, NodeJs, and Django

❖ **Database:**

- Database is the collection of inter-related data which helps in efficient retrieval, insertion and deletion of data from database and organizes the data in the form of tables, views, schemas, reports etc.
- Structured Database: SQL, MySQL, Postgresql, Oracle
- Non structured database: Mongodb

Technology Related to Full Stack Development

❖ API Integration:

- It is the structured way the frontend (Client) communicates with the backend (Server) and how the backend communicates with third-party services (Cloud).
 - RESTful APIs (Representational State Transfer): The industry standard. It uses standard HTTP methods (GET, POST, PUT, DELETE) and stateless communication. It is highly scalable and compatible with almost every cloud service.
 - GraphQL: Developed by Meta, this allows the frontend to request exactly the data it needs. This prevents "Over-fetching" (downloading too much data) and "Under-fetching" (needing multiple requests for one page), which is critical for mobile performance.
 - tRPC (TypeScript Remote Procedure Call): It allows the frontend to call backend functions directly with End-to-End Type Safety. If you change a database field on the server, the frontend code will immediately show a red error line.

Technology Related to Full Stack Development

❖ Caching & Database

- Database caching improves application performance by storing frequently accessed data in high-speed, in-memory storage (e.g., Redis, Memcached), reducing direct, slow queries to the database
 - Distributed Caching (Redis/Upstash): A global cache layer sits in front of the database. Frequent queries (like user sessions or product catalogs) are stored in-memory for microsecond response times.
 - The Global Edge: Logic is deployed to Edge Functions (Cloudflare Workers or Vercel Edge). This means your code runs in 300+ cities globally, so the "Backend" is physically closer to the user.
 - Database Evolution: Using Serverless Postgres (Neon) or Vector Databases (Pinecone). Vector DBs allow the application to perform "Semantic Search," enabling AI-driven features like "Find similar products."

Technology Related to Full Stack Development

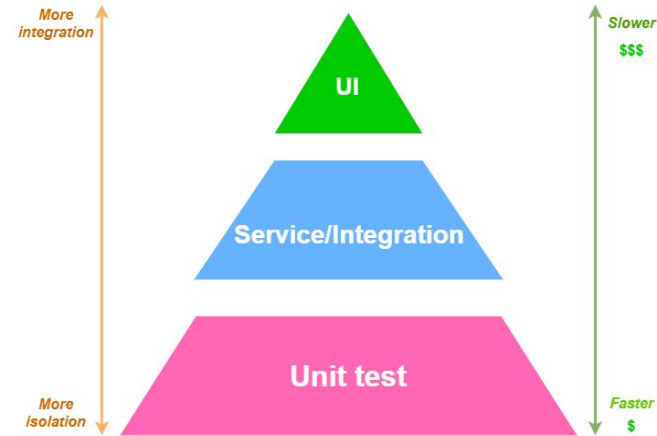
❖ Popular Stacks

- MERN Stack: MongoDB, Express, ReactJS and Node.js
- MEAN Stack: MongoDB, Express, AngularJS and Node.js.
- Django Stack: Django, python and MySQL as Database.
- Rails or Ruby on Rails: Uses Ruby, PHP and MySQL.
- LAMP Stack: Linux, Apache, MySQL and PHP.

Testing and Quality Assurance (QA)

❖ The Testing Pyramid: A Strategic Overview

- To build a resilient system, we distribute our testing efforts across three main tiers:
 - **Unit Tests** : Testing individual functions, components, or logic in total isolation.
 - Goal: Extreme speed and pinpointing the exact line of code that failed.
 - Tools: Vitest or Jest.
 - **Integration Tests**: Testing how different parts of the system work together (e.g., Does the API correctly save a user to the Database?).
 - Goal: Ensuring the "handshake" between modules is successful.
 - **UI / End-to-End (E2E) Tests**: Using a "Headless Browser" to mimic a real human clicking buttons, filling forms, and navigating the site.
 - Goal: Verifying the entire "Critical Path" (e.g., Can a user actually complete a purchase?).
 - Tools: Playwright or Cypress.



Unit Test

- ❖ A unit test is a small piece of code that checks if a specific function or method in an application works correctly.
- ❖ It will work as the function inputs and verifying the outputs.
- ❖ These tests check that the code work as expected based on the logic the developer intended.
- ❖ Strategy:
 - **Logic checks:** Verify if the system performs correct calculations and follows the expected path with valid inputs. Check all possible paths through the code are tested.
 - **Boundary checks:** Test how the system handles typical, edge case, and invalid inputs. For example, if an integer between 3 and 7 is expected, check how the system reacts to a 5 (normal), a 3 (edge case), and a 9 (invalid input).
 - **Error handling:** Check the system properly handles errors. Does it prompt for a new input, or does it crash when something goes wrong?
 - **Object-oriented checks:** If the code modifies objects, confirm that the object's state is correctly updated after running the code.

Integration Testing

- ❖ Integration Testing is a Software Testing Technique that focuses on verifying the interactions and data exchange between different components or modules of a Software Application.
- ❖ Importance:
 - Integration testing is important because it verifies that individual software modules or components work together correctly as a whole system.
 - This ensures that the integrated software functions as intended and helps identify any compatibility or communication issues between different parts of the system.
- ❖ Types:
 - Big-Bang Integration Testing:
 - All the modules of the system are simply put together and tested.
 - Practicable only for very small systems
 - An error is found during the integration testing, it is very difficult to localize the error as the error may potentially belong to any of the modules being integrated.

Integration Testing

❖ Types:

➤ Bottom-Up Integration Testing:

- The primary purpose of this integration testing is that each subsystem tests the interfaces among various modules making up the subsystem
- A principal advantage of this integration testing is that several disjoint subsystems can be tested simultaneously.
- It is easy to create the test conditions.
- It is Easy to observe the test results.

➤ Top-Down Integration Testing

- This is used in order to simulate the behaviour of the lower-level modules that are not yet integrated.
- Testing takes place from top to bottom.
- Separately debugged module.
- Few or no drivers needed.
- It is more stable and accurate at the aggregate level.
- Easier isolation of interface errors.

Integration Testing

❖ Types:

➤ Mixed Integration Testing:

- A mixed integration testing is also called sandwiched integration testing.
- a combination of top down and bottom-up testing approaches.
- Mixed approach is useful for very large projects having several sub projects.
- This Sandwich approach overcomes this shortcoming of the top-down and bottom-up approaches.
- Parallel test can be performed in top and bottom layer tests.
- This integration testing cannot be used for smaller systems with huge interdependence between different modules.

Automation Testing

- ❖ Automation Testing is a technique where the Tester writes scripts independently and uses suitable Software or Automation Tools to test the software.
- ❖ It is an Automation Process of a Manual Process.
- ❖ It allows for executing repetitive tasks without the intervention of a Manual Tester.
- ❖ Why Transform From Manual to Automated Testing?
 - Manual testing is a tedious task that can be boring and at the same time error-prone.
 - Automation testing is more efficient for detecting bugs in comparison to manual testing.
 - Manual testing requires huge manpower in comparison to automation testing which requires no human intervention and the test cases can be executed unattended.
- ❖ What Kinds of Software Tests Should Be Automated First?
 - End-to-End tests
 - Unit tests
 - Integration tests
 - Performance tests

CI/CD

- ❖ Also known as Continuous Integration and Continuous Delivery/Deployment CI/CD
- ❖ A practice that automates the process of building, testing, and releasing software.
- ❖ It forms the backbone of modern DevOps by connecting development and operations to deliver applications faster and more reliably.
 - It automates code integration, testing, and deployment workflows.
 - Reduces manual effort while improving software quality.
 - Enables faster, more consistent, and reliable releases.
- ❖ Previously, Developers used to create separate feature branches and worked on them for weeks or even months. All branches were merged at the end, which often caused big conflicts and broken builds.

The Three Pillars of CI/CD

❖ Continuous Integration (CI)

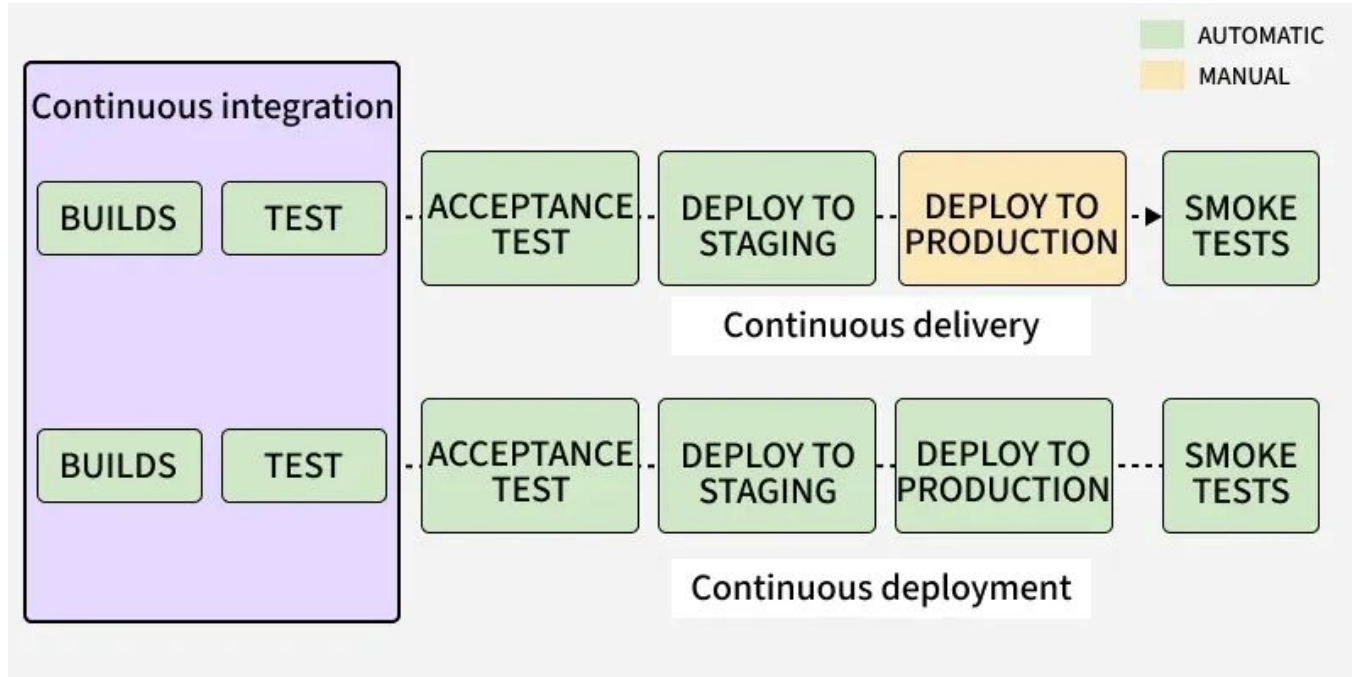
- The Goal: Prevent "Integration Hell."
- The Process: Developers **merge their changes back to the main branch** as often as possible (daily).
- The Automation: Upon every commit, an automated system builds the code and runs unit tests. If a test fails, the build is rejected, and the developer is notified immediately.

❖ Continuous Delivery (CD)

- The Goal: Be ready to release at any moment.
- The Process: After CI passes, the **code is automatically deployed to a staging/testing environment**. Extensive integration and load tests are run.
- The Release: The codebase is always in a deployable state, but the push to production is a manual decision (e.g., a "Deploy" button click by a manager).

❖ Continuous Deployment (CD)

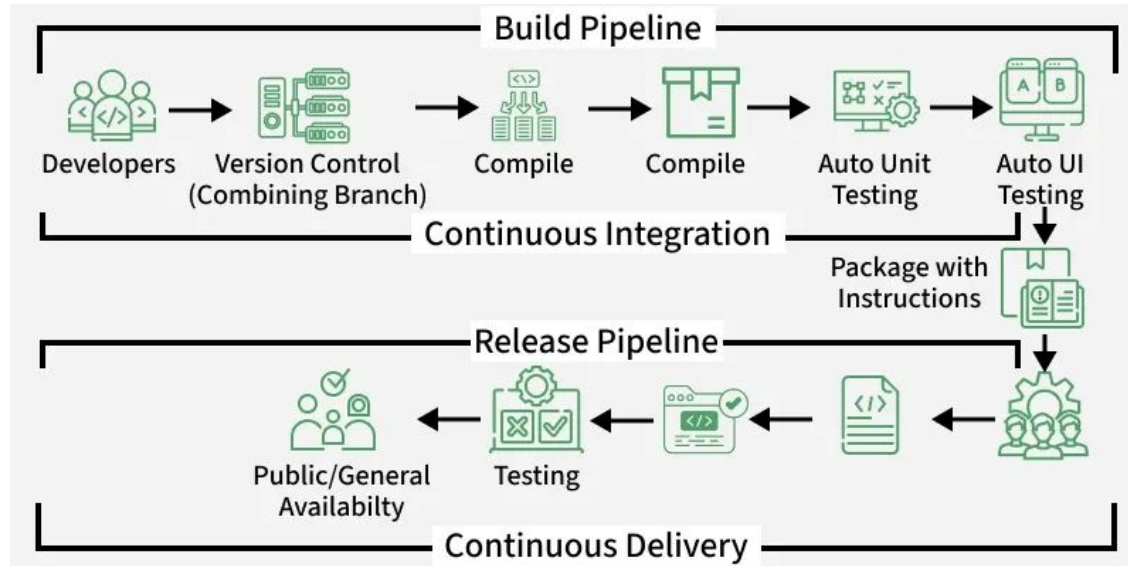
- The Goal: Eliminate human intervention.
- The Process: Same as Continuous Delivery, but if all automated tests pass in staging, the code is **automatically deployed to production**.
- The Requirement: Requires extremely robust, trustworthy automated testing suites



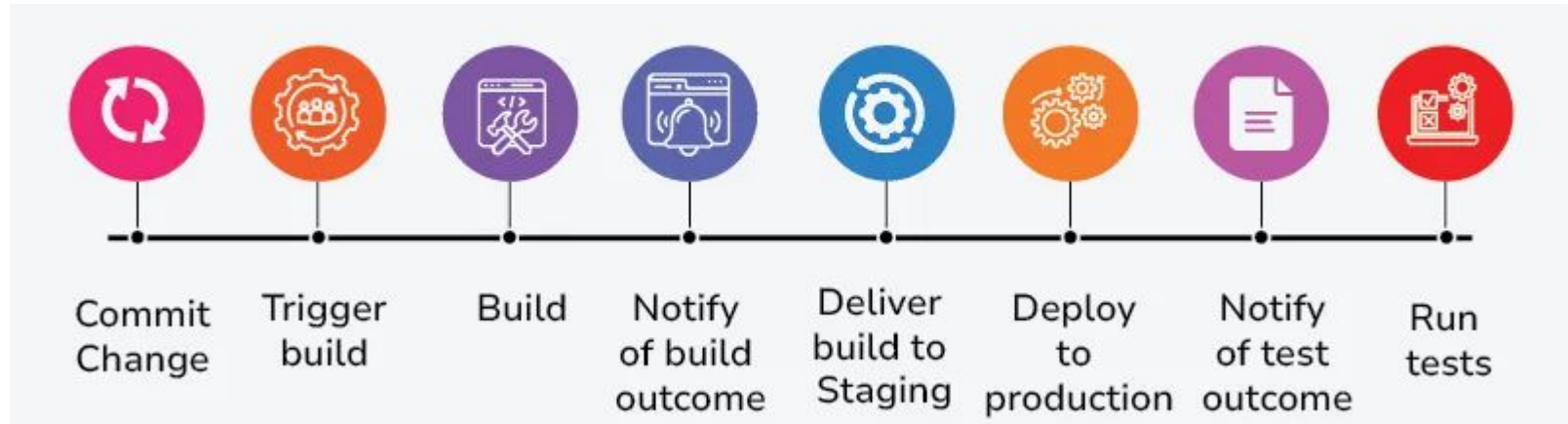
CI and CD Workflow

What is CI/CD Pipeline?

- ❖ **CI/CD (Continuous Integration and Continuous Delivery):** This is a way of working where developers can make many code changes and deliver updates quickly.
- ❖ **CI/CD Pipeline:** This is the actual step-by-step process that takes the code and automatically moves it from development to live production.



Components of a CI/CD Pipeline



GitHub Actions

- ❖ GitHub Actions is an automation tool built inside GitHub that helps you automatically run tasks like testing code, deploying applications, formatting files, sending notifications, etc., whenever something happens in your repository.
- ❖ GitHub Actions Workflow Components:
 1. **Events**
 - ❖ These events are specific activities in your repository that lead to the execution of actions.
 - Push: Triggered when code is pushed to the repository.
 - Pull Request: Activated when a pull request is created, opened, or updated.
 - Release: Occurs when a release is created or updated.
 - Label: Happens when a label is applied or removed.
 - Issues: Fired when an issue is opened, edited, or closed.
 - Milestone: Triggered when a milestone is added or updated.

- ❖ **Commit Change:** Developers make code changes and commit these changes to a version control system (e.g., Git).
- ❖ **Trigger Build:** The version control system detects the new commit and triggers the build process automatically.
- ❖ **Build:** The codebase is compiled and built into a deployable artifact, such as a binary executable or a Docker image. The build step verifies that the code compiles correctly and that all dependencies are resolved.
- ❖ **Notify of Build Outcome:** The CI/CD system notifies the team of the build results, whether it passed or failed. Immediate feedback on the build status helps developers quickly identify and resolve any build issues.
- ❖ **Run Tests:** Running tests ensures that the new code does not introduce any bugs or regressions and that it meets the required quality standards.
- ❖ **Notify of Test Outcome:** The results of the test suite are reported back to the development team.
- ❖ **Deliver Build to Staging:** If tests pass, the build artifact is deployed to a staging environment. The staging environment simulates the production environment, allowing for final validation before production deployment.
- ❖ **Deploy to Production:** After successful validation in staging, the build is automatically or manually promoted to the production environment. This step makes the new features and fixes available to end-users.

GitHub Actions

❖ Jobs in GitHub Actions

- Jobs are tasks that run within a GitHub Action workflow. By default, jobs run in parallel, but you can define dependencies so that one job runs after another. This is useful when certain tasks depend on the output of previous ones, such as building an app before running tests.

❖ Runners in GitHub Actions

- Runners are the virtual machines or environments where your GitHub Actions workflows execute. GitHub provides runners for various operating systems, such as Ubuntu, Windows, and MacOS.

❖ Managing Secrets

- You must never hardcode sensitive values like API keys, passwords, or cloud credentials in your YAML files. GitHub Actions provides Encrypted Secrets for this.
- How: You store them in your repository settings (Settings > Secrets and variables > Actions).
- Usage: You access them in your workflow using the secrets context:

GitHub Actions

❖ Self-Hosted Runners

- While GitHub-hosted runners are easy, self-hosted runners are a key system design choice for many companies
- Importance: Security & Network Access, Custom Hardware, Cost & Performance

❖ Caching Dependencies

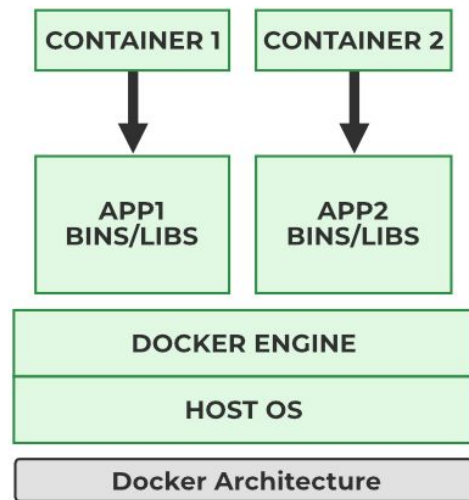
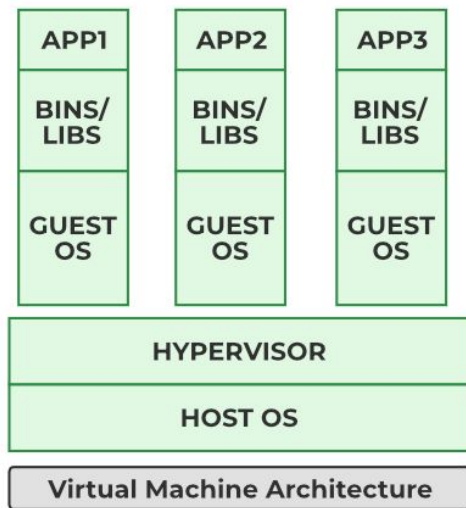
- CI/CD jobs run in clean environments, meaning they have to download dependencies (like `node_modules` or Maven packages) every single time. This is slow and wasteful.

❖ Matrix Builds

- To ensure your system is reliable, you might need to test it across multiple platforms, language versions, or configurations. A matrix strategy lets you run the same job multiple times with different parameters.

Docker

- ❖ Docker is an OS-level virtualization (or containerization) platform, which allows applications to share the host OS kernel instead of running a separate guest OS like in traditional virtualization.
- ❖ This design makes Docker containers lightweight, fast, and portable, while keeping them isolated from one another.



Components of Docker

- ❖ Docker Engine: Docker Engine has a core part docker daemon , that handles the creation and management of containers.
- ❖ Docker Image: Docker Image is a read-only template that is used for creating containers, containing the application code and dependencies.
- ❖ Docker Hub: It is a cloud based repository that is used for finding and sharing the container images.
- ❖ Dockerfile: It is a file that describes the steps to create an image quickly.
- ❖ Docker Registry : It is a storage distribution system for docker images, where you can store the images in both public and private modes.

Progressive Web App (PWA)

- ❖ A Progressive Web App (PWA) is an **application software delivered through the web**, built using common web technologies including HTML, CSS, and JavaScript.
- ❖ It is intended to **work on any platform that uses a standards-compliant** browser, including both desktop and mobile devices.
- ❖ Benefits of Progressive Web Apps
 - **Offline Usability**
 - **Page Loading Speed**
 - **Installable**
 - **Automatic Background Sync**
 - **SEO (Search Engine Optimization)**
 - **Platform Independence**

Components of a Progressive Web App

❖ Service Worker

- A Service Worker is a JavaScript file that acts as a **proxy between the browser and the network**.
- It enables **offline functionality and improves performance through efficient caching**. Service Workers run in the background, independent of the web page, allowing them to intercept network requests, cache or retrieve resources from the cache, and deliver push notifications.
- They enhance the reliability and performance of PWAs by managing network requests and providing offline access.

❖ Web App Manifest

- The Web App Manifest is a JSON file that **provides the browser with information about your web application**.
- This file includes details like the app's name, icons, theme colors, and start URL.
- The manifest file enables the PWA to be installed on the device's home screen and defines how it should appear to users. It allows developers to customize the app's appearance and behavior, ensuring a native-like experience.

Components of a Progressive Web App

❖ HTTPS

- PWAs require a secure protocol (HTTPS) because Service Workers can intercept network requests and modify responses, **ensuring secure and reliable functionality**.
- HTTPS ensures that all communications between the browser and the server are encrypted, protecting user data and enhancing security.
- It is a fundamental requirement for implementing Service Workers and other advanced features of PWAs.

Progressive Web App Work

❖ Initial Load and Installation

- When a user first visits a PWA-enabled website, the site loads like any other website.
- However, the presence of a Web App Manifest allows the user to install the app onto their device's home screen.
- This installation process does not involve downloading a large file from an app store; instead, it adds a shortcut to the home screen and caches essential resources for offline use.

❖ Caching and Offline Functionality

- Service Workers play a crucial role in enabling offline functionality.
- During the initial visit, the Service Worker is registered and installed in the background. It pre-caches key resources like HTML, CSS, JavaScript, and images.
- When the user revisits the site or interacts with the PWA offline, the Service Worker serves the cached resources, ensuring fast load times and offline access..

Progressive Web App Work

❖ Handling Network Requests

- Service Workers intercept network requests made by the PWA.
- They determine whether to serve a cached version of the resource or fetch it from the network.
- This approach enhances performance by reducing reliance on network connectivity and improving load times. If the network is available, the Service Worker can update the cache with the latest version of the resources.

❖ Push Notifications and Background Sync

- PWAs can send push notifications to re-engage users and provide timely updates.
- Service Workers handle these notifications, even when the PWA is not actively running.
- Additionally, PWAs can perform background synchronization, ensuring that user actions taken offline are synchronized with the server once the network is available.
- This feature enhances user experience by providing seamless interaction regardless of connectivity

Responsive Web Design

- ❖ Responsive Web Design ensures websites automatically adapt to different screen sizes, providing a smooth and user-friendly experience across all devices.
 - Content adapts by resizing or hiding based on the device.
 - Improves user experience and reduces user drop-off.
 - Uses a single flexible layout.
 - Implemented using HTML and CSS, including HTML5 and CSS3.
- ❖ Importance:
 - Prevents loss of user interest caused by poor display on different screen sizes.
 - Ensures accessibility for users across various devices and platforms.
 - Helps brands reach a wider audience and attract more customers.
 - Strengthens brand identity through consistent performance on all devices.
 - Enhances customer satisfaction regardless of how the website is accessed.

Principles of Responsive Web Design

❖ Viewable on Multiple Devices

- Ensure the website works properly on mobiles, tablets, and desktops.
- Design layouts that adapt to different screen sizes automatically.

❖ Easy to Understand

- Keep the layout clean and well-organized for better usability.
- Arrange content and features clearly to avoid user confusion.

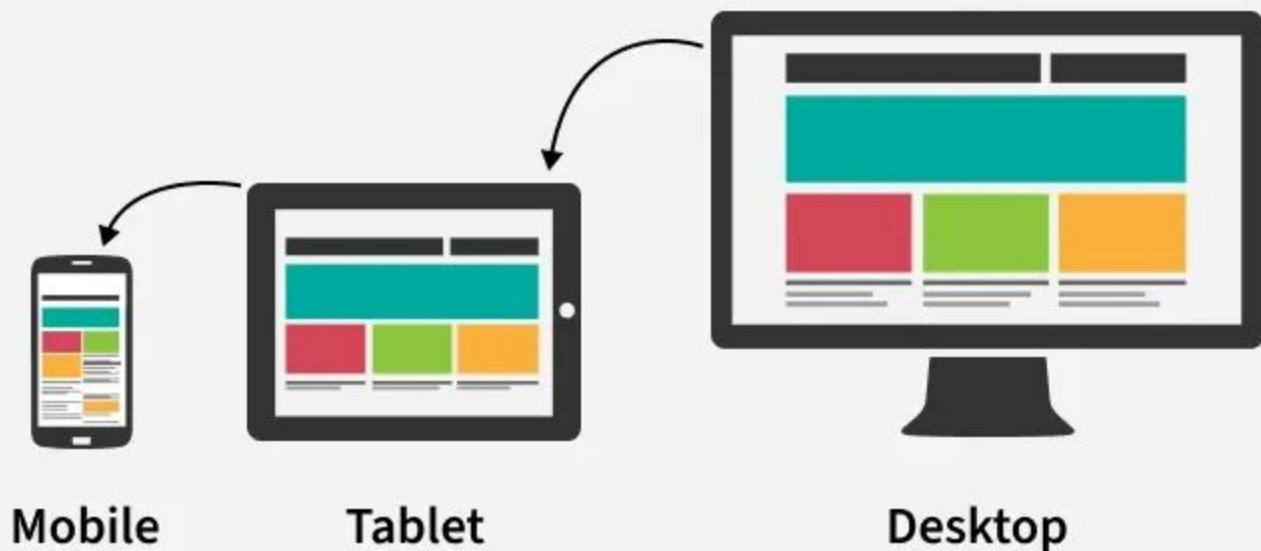
❖ Consistency

- Maintain the same fonts, colors, and design elements across devices.
- Provide a uniform experience whether viewed on phone, tablet, or PC.

❖ User Engagement

- Enable interactive features suited to each device, such as swipe gestures on mobile.
- Make navigation intuitive to improve user satisfaction and trust.

Responsive Web Design



Thank You